

Цель работы

Изучить полный цикл создания базы данных: от концептуального проектирования до физической реализации и написания прикладного кода на Python для взаимодействия с БД.

Этап 1. Проектирование (Аналитическая часть)

1.1. Описание предметной области

Вы разрабатываете систему для городской библиотеки. Библиотека содержит множество книг. Каждая книга имеет одного или нескольких авторов, издательство, год выпуска и уникальный инвентарный номер. Читатели могут регистрироваться в системе (ФИО, дата рождения, номер телефона, дата регистрации). Читатель может взять книгу на определенный срок (до 30 дней). Одна книга может быть выдана только одному читателю в данный момент (экземпляр уникален). Система должна вести учет выдач и возвратов (дата выдачи, плановая дата возврата, фактическая дата возврата). Если книга возвращена позже срока, начисляется штраф (условная сумма за каждый день просрочки).

1.2. Требования к этапу (сдать в виде отчета .docx или .pdf):

1. Концептуальная модель (ER-диаграмма):

- **Определить сущности, их атрибуты и связи (обозначить первичные и внешние ключи).**
- **Указать типы связей (1:1, 1:M, M:M).**

2. Логическая модель:

- **Переход к реляционной модели (таблицы).**
- **Нормализация до 3-й нормальной формы (3НФ) (кратко объяснить, почему таблицы находятся в 3НФ, и как были устранены транзитивные зависимости, если они были).**

3. Даталогическая модель (SQL-скрипт создания):

- Написать скрипты CREATE TABLE для PostgreSQL/MySQL (выбрать одну СУБД).
- Определить типы данных (VARCHAR, DATE, DECIMAL, INT, SERIAL/AUTO_INCREMENT).
- Прописать ограничения (PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK - например, чтобы год выпуска был не больше текущего, дата возврата >= даты выдачи).

Этап 2. Создание БД и наполнение тестовыми данными

Задачи:

1. Развернуть локальную БД (рекомендуется PostgreSQL или SQLite для упрощения настройки в Python).
2. Выполнить созданные в Этапе 1 SQL-скрипты.
3. Написать скрипт на Python (populate_db.py), который заполняет таблицы тестовыми данными (минимум: 5 книг, 3 автора, 4 читателя, 7 записей о выдаче, включая просроченные).

Этап 3. Разработка классов Python (ООП)

Напишите скрипт library_system.py, содержащий классы для взаимодействия с БД.

Структура классов:

1. Класс Database (Синглтон или просто класс с методами):
 - __init__(self, db_name): подключение к БД (использовать sqlite3 или psycopg2 для PostgreSQL).

- **execute_query(sql, params):** универсальный метод выполнения запросов.
- **fetch_all(sql, params):** метод получения данных.

2. Класс Book (Репрезентация строки таблицы книг):

- **Атрибуты:** id, title, publisher, year, inventory_number.
- **Метод save():** обновить данные в БД (UPDATE).
- **Метод delete():** удалить книгу.
- **Статический метод find_by_author(author_name):** найти все книги автора.
- **@property для is_available:** возвращает True, если книга не выдана.

3. Класс Reader:

- **Атрибуты:** id, full_name, birth_date, phone, reg_date.
- **Метод get_debt_books():** список книг, которые читатель должен вернуть (просроченные).

4. Класс Lending (Выдача):

- **Атрибуты:** id, book_id, reader_id, issue_date, due_date, return_date.
- **Метод close_lending(actual_return_date):** закрыть выдачу, рассчитать штраф (если есть) и записать в отдельную таблицу Fines.

Этап 4. Разработка консольного приложения (CLI)

Создайте файл main.py с меню для пользователя (администратора библиотеки).

Функционал меню:

- 1. Показать все книги (с указанием статуса: "В наличии" или "Выдана").**

- 2. Добавить нового читателя (ввод данных с клавиатуры, проверка на дубликаты).**
- 3. Выдать книгу читателю (поиск свободной книги по названию, проверка, не превышает ли читатель лимит книг (макс. 3)).**
- 4. Принять возврат книги (по инвентарному номеру). Расчет штрафа, если книга просрочена (штраф = 10 руб. за день).**
- 5. Показать должников (читатели, у которых есть просроченные книги).**
- 6. Поиск книги по автору (ввод фамилии автора).**
- 7. Выход.**